

Problem Editorial for Pecking Order.

CITS3001 Project 2026 — Group 35 — Problem D — 35-23669907

Overview

This problem is a topological sort with a uniqueness check. Each log entry “ $a\ b$ ” is a directed edge $a \rightarrow b$, and a ranking consistent with the log is exactly a topological order of that graph. The solver has to decide which of three cases holds: the graph has a cycle (no order, **CONTRADICTORY**), the graph is a DAG with more than one topological order (**AMBIGUOUS**), or it has exactly one order (print it). The technique is Kahn’s algorithm from the graph algorithms lectures (BFS-style topological sorting), with one extra observation.

Key insights

1. **A choice means ambiguity.** Kahn’s algorithm keeps a pool of swans that no remaining swan outranks. If the pool ever holds two swans u and w , either can go next, so (if there is no cycle) at least two rankings exist. If the pool always holds exactly one swan, every step is forced and the ranking is unique. Equivalently, the order is unique iff consecutive swans in it are joined by an edge (the DAG has a Hamiltonian path).
2. **Cycles beat choices.** A swan on a cycle never reaches in-degree 0, so Kahn’s stops early. This must be checked *after* the full run: a log with a choice point *and* a cycle is **CONTRADICTORY**, not **AMBIGUOUS**.
3. **Duplicates are harmless** as long as every copy is both counted in the in-degree and removed from it.

The story deliberately suggests ranking by wins (out-degree), which fails on sample 1.

Solution

The only data structures are an adjacency list `adj` (duplicates kept), an in-degree array `indeg`, a FIFO queue `ready`, and the output list.

```
1: read  $N, M$ ; for each entry  $(a, b)$ : append  $b$  to  $adj[a]$ ;  $indeg[b] \leftarrow indeg[b] + 1$ 
2:  $ready \leftarrow$  queue of all  $v$  with  $indeg[v] = 0$ ;  $order \leftarrow []$ ;  $unique \leftarrow \text{true}$ 
3: while  $ready$  not empty do
4:   if  $|ready| > 1$  then  $unique \leftarrow \text{false}$  ▷ a choice exists; keep going
5:    $u \leftarrow ready.pop()$ ; append  $u$  to  $order$ 
6:   for  $v \in adj[u]$  do
7:      $indeg[v] \leftarrow indeg[v] - 1$ ; if  $indeg[v] = 0$  then push  $v$  to  $ready$ 
8: if  $|order| < N$  then print CONTRADICTORY
9: else if not  $unique$  then print AMBIGUOUS
10: else print  $order$ 
```

The second accepted solution builds any topological order and then checks that every consecutive pair $(order[i], order[i + 1])$ is in a hash set of edges.

Approaches that do not work

- **Sort by wins** (`wa_win_count`): out-degree ignores who beat whom. Fails sample 1.
- **Lexicographically smallest order** (`wa_lexicographic`): never reports **AMBIGUOUS**.
- **Stopping at the first choice** (`wa_early_exit`): misses a cycle later on (tests 07, 13).
- **Treating a cycle as ambiguous** (`wa_cycle_is_ambiguous`): fails sample 3.
- **Counting distinct predecessors** but decrementing per copy (`wa_distinct_indegree`): a swan becomes ready too early (tests 06, 11, 18).
- **Recursive DFS** (`rte_recursive_dfs`): correct logic, but a $2 \cdot 10^5$ -swan chain recurses $2 \cdot 10^5$ deep and hits Python’s limit of 1000.
- **All-pairs reachability** (`t1e_all_pairs`): the order is unique iff every pair is comparable, but one search per swan is $\Theta(N(N + M))$. Measured: 0.24s, 0.84s, 3.3s at $N = 2000, 4000, 8000$ — roughly $4\times$ per doubling, so about 4×10^{10} steps at the limit.

Complexity analysis

Time. Reading the input and building `adj` and `indeg` is $O(N + M)$. In Kahn's loop each swan enters `ready` at most once (only when its in-degree hits 0), so there are at most N pops, and each edge is visited exactly once when its tail is popped, giving M decrements. Queue operations are $O(1)$. Total: $O(N + M)$. The Hamiltonian variant adds $N - 1$ expected- $O(1)$ set lookups, still $O(N + M)$.

Space. `adj` stores M entries across N lists, and `indeg`, `ready` and `order` are each $O(N)$. Total: $O(N + M)$.

This agrees with the test data: at $N = M = 2 \cdot 10^5$ the reference solution runs in about 0.4s in CPython (0.7s for the Hamiltonian variant), well inside the 3s limit, while the quadratic solution would need thousands of seconds.

Test data

20 secret tests. Tests 01–09 are small hand-made cases: $N = 1$, $N = 2$ with no log, a 2-cycle, redundant shortcut edges, a duplicate entry, a choice followed by a cycle, a cycle with no choice before it, and ambiguity only at the last step. Tests 10–20 are at or near the limits (N, M up to $2 \cdot 10^5$): a shuffled full chain, a chain with redundant and duplicated edges, a random DAG, a random cyclic graph, ambiguity only at the very last step, a cycle only at the end, $N = 2$ with all M entries duplicated (one way, and both ways), a dense 1200-swan log, a star, and a chain with one missing link. Every non-accepted submission fails at least two secret tests. All tests are produced by the seeded `generators/gen.py` and checked by `input_validators/validate.py`.

Declaration of AI use

I used Claude (Anthropic) as a coding and writing assistant. It helped me restructure the package into `problemtools` format, convert the statement to a `\problemname` fragment, rewrite the comments in my solutions in my own commenting style, write the test generator and input validator, run `verifyproblem` to confirm every submission gets its intended verdict, and draft this editorial. I chose the problem idea and story, and I checked the solutions, the test data and the analysis above myself.